Daily Quant Digest: 2026-11-07

Foundations Restart, day 12. Unit A of probability is done, so today we open Unit B with **random variables**: how to turn “what happened” into a number, and how to describe where the probability sits. In derivatives we meet the most traded contract in the world, the **interest rate swap**. In algorithms we start Unit B with the **stack**, the data structure behind every “undo” button.

Today’s topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 11: random variables, the PMF and the CDF	Easy-Medium
Derivatives	FR Lesson 12: interest rate swaps	Easy-Medium
Algorithms & DS	FR Lesson 12: stacks: decode a nested string	Medium
Music	Bruce Springsteen, “Born to Run” (1975)	

---

1. Probability & Statistics: Random Variables, the PMF and the CDF

> **Where we are.** Lesson 1: the **sample space** is a room of possible worlds, and an event is a group of those worlds. Lessons 3 to 5: counting the worlds. Lessons 6 to 10: how news moves the walls. Until now every answer was “the chance of an event”. Today we attach a **number** to every world, which is what lets us later talk about averages, variance and risk.

1.1 The central picture: a price tag on every world

Imagine the room of possible worlds from Lesson 1. A **random variable** is a person who walks around the room and sticks a **price tag** (a number) on every world. That is all it is: a rule that gives each outcome a number.

Flip three coins. The room has 8 worlds: HHH, HHT, …, TTT. Let X be “the number of heads”. The person writes 3 on HHH, 2 on HHT, HTH and THH, and so on.

In the picture	In the maths
A world in the room	An outcome ω in the sample space
The price tag on it	The value $X(\omega)$
All worlds with tag 2	The event $\{X = 2\}$
Total weight of those worlds	$P(X = 2)$

Two things are worth saying out loud. First, a "random variable" is not random and not a variable. It is a fixed rule. The randomness is in which world we land in. Second, many worlds can share the same tag. HHT, HTH and THH are different worlds but all carry tag 2.

The same room can hold many different tags. On the three coin flips you could also tag "number of tails", or "1 if the first flip is heads, else 0", or "your P&L if you bet USD 10 on each flip". Same worlds, different numbers.

1.2 The PMF: sort the worlds into buckets by tag

Now line up buckets labelled 0, 1, 2, 3. Drop each world into the bucket matching its tag, and weigh each bucket. The weights are the **probability mass function (PMF)**:

$$p(x) = P(X = x).$$

Read it out loud: "p of x is the chance that the tag equals x". Lowercase x is a plain number (a bucket label). Capital X is the rule.

Count first. Imagine 8,000 traders, each flipping three coins. About 1,000 land in each of the 8 worlds. So:

Tag x	Worlds	Traders (of 8,000)	$p(x)$
0	TTT	1,000	$1/8$
1	HTT, THT, TTH	3,000	$3/8$
2	HHT, HTH, THH	3,000	$3/8$
3	HHH	1,000	$1/8$

Two rules always hold, because every world lands in exactly one bucket:

- every $p(x)$ is between 0 and 1;
- the weights add to 1: $\sum_x p(x) = 1$.

The symbol $\sum_x$ just means "add up over all bucket labels x". Here: $\frac{1}{8} + \frac{3}{8} + \frac{3}{8} + \frac{1}{8} = 1$.

1.3 The CDF: a running total, like a water tank filling up

The **cumulative distribution function (CDF)** answers a different question: "what is the chance the tag is **at most** x?"

$$F(x) = P(X \le x).$$

Picture walking along the number line from left to right and pouring each bucket into a tank as you pass it. $F(x)$ is the water level when you are standing at x.

x	$F(x)$
below 0	0
0 to just under 1	$1/8$
1 to just under 2	$4/8$
2 to just under 3	$7/8$
3 and above	1

The level only jumps when you pass a bucket, and the size of each jump is that bucket's weight. Between buckets it stays flat. So the graph is a **staircase**: it starts at 0, never goes down, and ends at 1. For example $F(2.5) = 7/8$: standing at 2.5 you have passed buckets 0, 1 and 2.

The two are the same information. From PMF to CDF: add up as you go. From CDF to PMF: the jump at x is $p(x)$. And any range is a difference of two water levels:

$$P(a < X \le b) = F(b) - F(a).$$

With numbers: $P(0 < X \le 2) = F(2) - F(0) = \frac{7}{8} - \frac{1}{8} = \frac{6}{8}$, which matches $p(1) + p(2)$.

1.4 Why bother with the CDF? The "everyone is above" trick

For some questions the CDF is far easier than the PMF. Take the **minimum** of several dice. "The smallest die is at least 4" means "**every** die is at least 4". For independent dice (Lesson 7) that is just a product. So we get the running total from the other side first, then read off each bucket as a difference. You will see this in the interview question below. Interviewers love it.

1.5 Common mistakes

- **Mixing up the tag and the world.** $P(X = 2) = 3/8$ because three different worlds carry the tag 2. Count worlds, not tags.
- $<$ **versus** $\le$**.** For a variable that only takes whole numbers, $P(X < 2) = F(1)$, not $F(2)$. On a staircase, the step at 2 belongs to "≤ 2" but not to "< 2".
- **Thinking the CDF only lives at the bucket labels.** F is defined for every real number. It is just flat between the jumps.

1.6 Self-check

1. Flip two coins. Write down the PMF of the number of heads.
2. For three coins, what are $F(-1)$, $F(1.7)$ and $F(10)$?
3. Could $p(0) = 0.6$ and $p(1) = 0.5$ be a PMF? Why not?

1.7 Interview question: dice tags

(A SIG / Optiver style warm-up. They want clean counting and the CDF trick for minima.)

Roll two fair dice. Let D be the **absolute difference** of the two numbers (so a 2 and a 5 give $D = 3$).

a. Find the PMF of D. Which value is most likely?

b. Find $P(D \le 2)$ and $P(1 \le D \le 3)$ using the CDF.

Now roll **three** fair dice and let M be the **smallest** of the three.

c. Find $P(M \ge k)$ for each k, then the full PMF of M.

d. A friend offers you a bet: you win if all three dice show 4 or more. What odds would make the bet fair?

1.8 Solution

(a) There are 36 equally likely worlds (Lesson 3). Count each difference. $D = 0$ needs a double: 6 worlds. $D = 1$: pairs like (1,2) and (2,1), with the smaller number from 1 to 5, so $5 \times 2 = 10$. In general, for $d \ge 1$ the smaller number can be $1, \dots, 6 - d$, and either die can be the smaller one: $2(6 - d)$ worlds.

d	**0**	**1**	**2**	**3**	**4**	**5**
worlds	6	10	8	6	4	2
$p(d)$	6/36	10/36	8/36	6/36	4/36	2/36
$F(d)$	6/36	16/36	24/36	30/36	34/36	36/36

Check: $6 + 10 + 8 + 6 + 4 + 2 = 36$. The most likely value is $D = 1$, not $D = 0$, because a difference of 1 can happen in two orders while a double cannot.

(b) $P(D \le 2) = F(2) = 24/36 = 2/3$. And $P(1 \le D \le 3) = P(0 < D \le 3) = F(3) - F(0) = 30/36 - 6/36 = 24/36 = 2/3$.

(c) $M \ge k$ means all three dice are at least k. Each die does that with chance $(7 - k)/6$, independently, so

$$P(M \ge k) = \left(\frac{7 - k}{6}\right)^3.$$

In counts out of 216: $k = 1, \dots, 6$ gives $216, 125, 64, 27, 8, 1$. Each bucket is the drop between neighbours: $P(M = k) = P(M \ge k) - P(M \ge k + 1)$.

k	**1**	**2**	**3**	**4**	**5**	**6**
$P(M \geq k)$ (of 216)	216	125	64	27	8	1
$P(M = k)$ (of 216)	91	61	37	19	7	1

Check: $91 + 61 + 37 + 19 + 7 + 1 = 216$.

(d) "All three at least 4" is $M \geq 4$: $27/216 = 1/8$. You win 1 time in 8, so fair odds are **7 to 1**: stake 1 to win 7.

Interview trick. For a max or min, never count the PMF directly. Write "everyone is below / above" as a product, then take differences.

Sanity check. $M = 1$ is the most likely minimum (91/216, about 42%), which makes sense: with three dice, at least one 1 is common ($1 - (5/6)^3$). All counts were checked by listing all 36 and 216 worlds by computer.

Self-check answers. (1) $p(0) = 1/4$, $p(1) = 1/2$, $p(2) = 1/4$. (2) 0, $4/8$, 1. (3) No: the weights add to 1.1, more than 1.

---

2. Derivatives: Interest Rate Swaps

> **Where we are.** Lesson 2: discount factors. Lesson 5: a forward locks in a price today. Lesson 8: a bond is fixed rent. Lesson 10: forward rates from the yield curve ("two roads to the same village"). Lesson 11: SOFR, the overnight repo rate. Today we combine them: a swap is a bundle of forwards on interest rates.

2.1 The scene

A Hong Kong property developer has borrowed **USD 100m for 3 years** from a bank at a **floating** rate: SOFR + 1.20%, reset every year. If US rates jump, its interest bill jumps, and its rental income does not. Its CFO would rather know the bill in advance.

On the other side sits a **pension fund**. It holds lots of floating-rate assets but owes fixed pensions, so it would like to *receive* a fixed rate. In between stands a **bank's swap desk**, quoting both sides all day.

The developer enters a **3-year interest rate swap**: it **pays fixed** 3.995% a year on USD 100m and **receives SOFR** on USD 100m. No USD 100m ever changes hands. Only the interest does.

2.2 The central picture: swapping electricity contracts

You have a flat whose electricity bill floats with the market price. Your neighbour has a fixed-price contract. You agree: “Each month, I pay you the fixed price, and you pay me whatever the market price turns out to be.” You still pay your floating bill, but your neighbour’s payment cancels it. Net, you pay the fixed price. You have **turned floating into fixed** without changing supplier.

In the picture	In the maths
The floating electricity bill	The loan’s SOFR payments
What you pay the neighbour	The **fixed leg**: $N \times K$ each year
What the neighbour pays you	The **floating leg**: $N\times$ SOFR each year
The size of the flat	The **notional** N, never exchanged
Only the difference changes hands	**Net settlement**

Where it stops: your neighbour might move away. A swap has **counterparty risk**, which today is mostly removed by central clearing (like futures in Lesson 7).

2.3 The mechanics

Names. The side paying fixed is the **payer** (or “long the swap”). The side receiving fixed is the **receiver**. The fixed rate K is the **swap rate**.

Payments. Each year, the payer receives $N \times (\text{floating} - K)$. If SOFR is above K, the payer gains; below, it pays. (Real USD swaps use compounded overnight SOFR, known only at the end of each period. To keep the picture simple, treat each year’s floating rate as one number.)

The fair swap rate. At the start, the swap must be worth zero to both sides, or nobody would agree. From Lesson 10, the curve gives a **forward rate** for each year: the rate you can lock in today for that future year. The fair fixed rate is the one where the fixed payments and the expected floating payments (at forward rates) have the same present value:

$$K \sum_i DF_i = \sum_i f_i \, DF_i \quad \Longrightarrow \quad K = \frac{\sum_i f_i \, DF_i}{\sum_i DF_i}.$$

Read it out loud: “the swap rate is an **average of the forward rates**, weighted by the discount factors”. Near years count a bit more, because their money is worth more today. The sum $\sum_i DF_i$ is called the **annuity**: the value today of receiving 1 a year.

Risk. A payer is like someone who has **sold a fixed bond** (Lesson 8): when rates rise, the fixed payments it has locked in look cheap, so the swap gains. Its **DV01** is about $N \times \text{annuity} \times 0.0001$.

2.4 Worked trade: the developer's swap

Illustrative zero rates (annual compounding): 1y 3.80%, 2y 3.90%, 3y 4.00%.

Step 1: discount factors. $DF_1 = 1/1.038 = 0.96339$, $DF_2 = 1/1.039^2 = 0.92634$, $DF_3 = 1/1.04^3 = 0.88900$. Annuity $= 2.77872$.

Step 2: forward rates (two roads, Lesson 10). $f_1 = 3.80\%$. $f_2 = DF_1/DF_2 - 1 = 4.00\%$. $f_3 = DF_2/DF_3 - 1 = 4.20\%$.

Step 3: the swap rate.

$$K = \frac{0.038(0.96339) + 0.040(0.92634) + 0.042(0.88900)}{2.77872} = \frac{0.111004}{2.77872} = 3.9948\%.$$

The desk quotes **3.995%**. It sits a touch below the simple average of 4.00%, because the low first year carries the most weight.

Step 4: payments at today's forwards.

Year	Fixed paid	Floating received (forward)	Net to payer	PV of net
1	3,994,770	3,800,000	−194,770	−187,640
2	3,994,770	4,000,096	+5,326	+4,934
3	3,994,770	4,200,289	+205,519	+182,706
Total				**0**

The developer pays a little in year 1 and gets it back later. Value at the start: zero, as it should be.

Step 5: what actually happens. Using $K = 3.995\%$, suppose SOFR turns out to be 3.80%, then 4.50%, then 3.00%.

Year	Loan interest (SOFR + 1.20%)	Swap net received	Total cost
1	5,000,000	−195,000	5,195,000
2	5,700,000	+505,000	5,195,000
3	4,200,000	−995,000	5,195,000

Whatever SOFR does, the developer pays **5.195%** (3.995% + 1.20%) every year. In year 3 it "loses" on the swap, but that is the point: it bought certainty, not a bet.

Step 6: mark-to-market. DV01 $= 100{,}000{,}000 \times 2.77872 \times 0.0001 \approx$ **USD 27,800 per bp**. If the whole curve rises 50 bp the day after the trade, repricing gives the payer a gain of about **USD 1.37m**; a 50 bp fall gives a loss of about USD 1.40m. The gain and loss are not equal: a small taste of convexity (Lesson 9).

2.5 When it went wrong: Hammersmith and Fulham

In the late 1980s the London Borough of Hammersmith and Fulham, a local council, built up a huge book of interest rate swaps and options, far bigger than its own borrowing, effectively betting that UK rates would fall. Rates rose sharply in 1988 and 1989 and the council faced large losses. Then the courts were asked whether a council was even allowed to trade swaps. In 1991 the House of Lords ruled that it was not: the contracts were beyond the council's legal powers and therefore **void**. The banks on the other side, not the council, ended up absorbing the damage. The lesson for traders: know your counterparty, including whether it has the legal power to sign the trade.

2.6 Likely interview questions

1. **"A company has a floating loan and fears rising rates. Pay or receive fixed?"** Pay fixed, receive floating. The floating receipts cancel the loan's floating payments.
2. **"Why is the swap rate close to, but not equal to, the average forward rate?"** It is a discount-factor-weighted average, so near-term forwards weigh more. With an upward-sloping curve, the swap rate sits slightly below the plain average.
3. **"You receive fixed on USD 50m 5-year at a DV01 of about USD 23,000. Rates fall 10 bp. Roughly?"** A receiver gains when rates fall: about +USD 230,000.

Self-check. (1) Does the notional change hands? (2) After a payer swap is struck, rates rise. Is it an asset or a liability for the payer? (3) What is the annuity in words?

Answers. (1) No, only net interest. (2) An asset: it pays a fixed rate now below the market. (3) The value today of receiving 1 each year for the life of the swap.

---

3. Algorithms & Data Structures: Stacks and the Nested String

> **Where we are.** Lesson 2: `string` and `vector`. Lesson 9: recursion and the call stack, a chain of helpers each holding a sticky note. Today we build that chain ourselves, by hand, with `std::stack`.

3.1 The central picture: a pile of paused jobs

You are working on a report. Your boss walks in with an urgent task. You put the report face down on your desk and start the new task. Then a colleague interrupts with something even more urgent. You put the urgent task on top of the report, and start the colleague's job. When you finish it, which job do you go back to? The one **on top of the pile**: the boss's task. Then the report.

That pile is a **stack**. You can only touch the top: **push** (put a job on top), **top** (look at it), **pop** (take it off). The last thing in is the first thing out (**LIFO**).

In the picture	In the code
Putting a job down to start a new one	`push`
Going back to the most recent job	`top` then `pop`
The job in your hands	the current state (`cur`, `k`)
The pile being empty	`empty()`

Stacks appear whenever things **nest**: brackets inside brackets, function calls inside function calls, the browser's back button, "undo". Every push and pop is $O(1)$.

3.2 The problem: decode a nested string

An encoded string uses the rule `k[text]`, meaning "`text` repeated `k` times". Rules can nest. Decode it.

- Input: `3[a]2[bc]`. Output: `aaabcbc`.
- Input: `3[a2[c]]`. Output: `accaccacc` (inside first: `2[c]` is `cc`, so `acc` three times).
- Input: `2[abc]3[cd]ef`. Output: `abcabccdcdcdef`.

Constraints: $1 \le |s| \le 30$; lowercase letters, digits and brackets only; the input is always valid; $1 \le k \le 300$ (so k can have several digits); the output has at most 10^5 characters.

3.3 Brute force first

Find the first `]`. The nearest `[` before it is an **innermost** bracket (nothing nested inside). Read the number before it, expand that piece, and rebuild the whole string. Repeat until no brackets are left.

`3[a2[c]]` becomes `3[acc]`, then `accaccacc`. It works, but every expansion copies the whole string. With b bracket pairs and an output of length L, that is about $O(b \times L)$ work, and lots of searching. The file includes it as a checker.

3.4 The insight: read left to right, pausing at every `[`

Read the string once, keeping the "job in your hands": the text `cur` built so far at this level, and the number `k` being read.

- **A digit**: extend the number. Reading `1` then `2` gives $1 \times 10 + 2 = 12$.
- `[`: a new nested job starts. Push the pair (`cur`, `k`) on the stack: "here is what I had, and how many times the coming block must repeat". Then start fresh: `cur = ""`, `k = 0`.
- **A letter**: append it to `cur`.
- `]`: the inner job is done. Pop (`before`, `times`). The new `cur` is `before` followed by `cur` repeated `times` times.

Trace `3[a2[c]]`:

Read	Action	Stack (bottom to top)	`cur`
`3`	$k = 3$		“”
`[`	push (“”, 3)	(“”,3)	“”
`a`	append	(“”,3)	“a”
`2`	$k = 2$	(“”,3)	“a”
`[`	push (“a”, 2)	(“”,3), (”a”,2)	“”
`c`	append	(“”,3), (”a”,2)	“c”
`]`	pop (“a”,2): “a” + “cc”	(“”,3)	“acc”
`]`	pop (“”,3):”” + “acc” x3		“accaccacc”

Complexity. Each input character is handled once, and each output character is written a bounded number of times per nesting level. Time is $O(|s| + L \cdot d)$, where d is the nesting depth, and in practice about $O(L)$. Space: $O(L)$ for the strings plus a stack of at most d frames.

3.5 C++ solution

```cpp
#include <cctype>
#include <stack>
#include <string>
#include <utility>
using namespace std;

string decodeString(const string& s) {
    stack<pair<string, int>> paused;   // (text before '[', repeat count)
    string cur;                        // the text of the level we are in
    int k = 0;                         // the number being read, digit by digit
    for (char c : s) {
        if (isdigit(static_cast<unsigned char>(c))) {
            k = k * 10 + (c - '0');     // "12" -> 1, then 1*10+2 = 12
        } else if (c == '[') {
            paused.push({cur, k});      // pause the outer job
            cur.clear();                // start a fresh inner job
            k = 0;
        } else if (c == ']') {
            auto [before, times] = paused.top();   // resume the outer job
            paused.pop();
            string block;
            block.reserve(cur.size() * times);
            for (int i = 0; i < times; ++i) block += cur;
            cur = std::move(before);
            cur += block;
        } else {
            cur += c;                   // an ordinary letter
        }
    }
    return cur;
}
```

The full file `2026-11-07.cpp` adds the brute force, seven examples (including multi-digit `10[x]` and triple nesting), and a 100,000-case random cross-check against the brute force. It compiles cleanly with `g++ -std=c++17 -O2 -Wall` and all tests pass.

3.6 Walkthrough of the C++

`stack<pair<string, int>> paused;` `std::stack` (header `<stack>`) is an **adapter**: it wraps another container (by default a `deque`) and only lets you use the top. Its member functions are `push`, `top`, `pop`, `empty` and `size`. Each element is a `std::pair` (header `<utility>`), a tiny box holding two values, here `.first` (a string) and `.second` (an int).

`paused.push({cur, k});` The braces build a `pair<string,int>` on the spot. This **copies** `cur` into the stack, which is what we want, because we clear `cur` on the next line. (`emplace(cur, k)` would build the pair directly inside the stack; same result here.)

`isdigit(static_cast<unsigned char>(c))`. `isdigit` (from `<cctype>`) expects a value that fits in an `unsigned char`. A plain `char` can be negative on some systems, and passing a negative value is undefined behaviour. The cast is the standard safe habit (we met the same idea with array indexing in Lesson 8).

`k * 10 + (c - '0')`. Characters are small numbers, and `'0'` to `'9'` are consecutive. So `'7' - '0'` is 7. Multiplying by 10 first shifts the old digits left, like reading a number on an odometer.

`auto [before, times] = paused.top();` A **structured binding** (C++17, Lesson 7): it unpacks the pair into two named variables. `top()` returns a reference to the top element; because we wrote `auto` and not `auto&`, the pair is copied into `before` and `times`. That matters: the next line `pop()` destroys the top element, so a reference to it would dangle. **Rule: never keep a reference to `top()` across a `pop()`.** Calling `top()` or `pop()` on an empty stack is undefined behaviour; valid input guarantees every `]` has a matching `[`.

`block.reserve(cur.size() * times);` Reserves memory once, so the repeated `+=` never has to grow and copy (Lesson 6's `reserve`, on a string this time).

`cur = std::move(before);` `std::move` says "I am done with `before`, take its memory instead of copying it". Then `cur += block` appends. The order matters: the outer text comes first.

`for (char c : s)`. A range-based loop over the characters. `s` is passed as `const string&`: no copy, read only.

Common bugs. Forgetting that k can have several digits (reading only one). Forgetting to reset `k = 0` after a `[`. Appending in the wrong order (`block + before`). Returning a reference to a local string.

Self-check. (1) Decode `2[x3[y]]`. (2) What is on the stack just after reading the second `[` of `2[a2[b2[c]]]`? (3) Why can we not use `auto&` for the structured binding here?

Answers. (1) `xyyyxyyy`. (2) ("", 2) at the bottom and ("a", 2) on top. (3) `pop()` destroys the element the reference would point to.

---

4. Music: Bruce Springsteen, “Born to Run” (1975)

Context

“Born to Run” is the title track of **Bruce Springsteen’s** third album, released in **August 1975**. Springsteen wrote it, and he recorded it with the **E Street Band**. His first two albums (both 1973) had been praised by critics but sold poorly, and his record company, Columbia, was losing patience. This album was his make-or-break moment.

The story behind it

Springsteen spent months on the title song alone, recording it in 1974 before most of the album existed. He wanted it to sound enormous, in the spirit of Phil Spector’s “Wall of Sound” records from the early 1960s, with guitars, saxophone, piano, glockenspiel and strings stacked on top of each other. He has described the song as about two young people trying to escape a dead-end town before it traps them. When the album finally came out it was a hit, and in October 1975 Springsteen appeared on the covers of both **Time** and **Newsweek** in the same week, something almost unheard of for a rock musician.

Why people love it

It sounds like running itself: urgent, breathless and hopeful. It captures the feeling of being young and sure that life is waiting somewhere else, which almost everyone has had. It has become Springsteen’s signature song and a centrepiece of his famously long live shows.

What to listen for

- **The opening.** Drums and a surging guitar riff hit at full speed from the first second. There is no warm-up.
- **The glockenspiel.** A bright, chiming bell sound rides on top of the band. It is the signature of the Spector-style sound he was chasing.
- **Clarence Clemons’ saxophone.** The big tenor sax solo is one of the most famous in rock.
- **The breakdown and the count-in.** Midway through, the song drops away almost to nothing, then Springsteen counts the band back in and everything crashes back louder than before. Listen for how the tension builds before that moment.

Next: Sunday (2026-11-08) is a revision day for all three tracks. Monday continues with Lesson 12 in probability (expected value as a balance point), Lesson 13 in derivatives (Treasury futures and cheapest-to-deliver), and Lesson 13 in algorithms (queues and deques).